

# A3-PS026-TC249

AT URI Syntax Compatibility and Round-Trip Validator

**Y Pavan Charan**
**pavancharany@gmail.com**
**R Hari Sai**
**hari.sai1796r@gmail.com**

AIORI Hackathon 3
October 14, 2026

# Problem Description

- AT URIs identify records in decentralized networks like Bluesky.
- Different parsers may interpret AT URIs differently, causing compatibility issues.
- Generic URI rules may differ from AT Protocol-specific rules.
- Reversible transformations may alter or lose important identifier components.
- A validator is needed to compare two parsers, test two reversible transformations, and ensure identifiers are preserved through round trips.

# Background

- AT Protocol supports decentralized social networking applications such as Bluesky.
- AT URIs (`at://`) are used to identify records within the AT Protocol ecosystem.
- An AT URI consists of an authority, collection identifier, and record key.
- Different software parsers may interpret certain identifiers differently.
- Transformations may unintentionally change or lose important identifier components.
- Parser comparison and round-trip testing help detect compatibility issues and preserve identifiers.

Example of an AT URI:
`at://alice.example/app.bsky.feed.post/123'
- Authority: `alice.example`
- Collection: `app.bsky.feed.post`
- Record Key:`123`

# Literature & RFC Review

- **AT URI Scheme Specification:** Defines the structure and rules of `at://` identifiers used in the AT Protocol.
- **RFC 3986:** Defines the generic syntax for URIs, including scheme, authority, and path components.
- **Parsing Compatibility:** AT URIs follow some generic URI rules but also have specific requirements, particularly for DID authorities containing colons.
- **Normalization and Encoding:** URI standards explain how reserved characters and percent-encoding can affect identifier interpretation.
- **Our Approach:** Compare two parsers and test reversible transformations to identify compatibility issues and preserve identifier components.

# Proposed Solution

- Develop a validator to check AT URI syntax using AT Protocol-specific rules and compare it with RFC 3986 parsing.
- Create a dataset of 100+ valid, invalid, and edge-case AT URIs.
- Implement two independent parsers to identify differences in URI interpretation.
- Apply two reversible transformation methods and verify whether the original identifier is restored.
- Check the preservation of important components, including authority, collection, and record key.
- Generate automated test results highlighting parser mismatches, validation errors, and transformation failures.
- Produce a compatibility report and demonstrate the working prototype to the mentor.

# Optimization Proposed

| BEFORE OPTIMIZATION | AFTER OPTIMIZATION |
|---|---|
| Basic URI validation | AT Protocol specification-based validation |
| Shared parser logic | Two independent parsers for comparision |
| Generic URI parsing | Compatibility checks against RFC 3986 |
| Basic encoding and round-trip tests | Two reversible transformations with component-preservation checks |
| Limited test cases and basic errors | 100+ test cases with detailed error explanations |
| Simple CSV output and syntax checks | Compatibility report distinguishing syntax validity from protocol resolution |

# Solution Architecture & Design

**Visual: Flowchart or diagram showing the optimization process.**

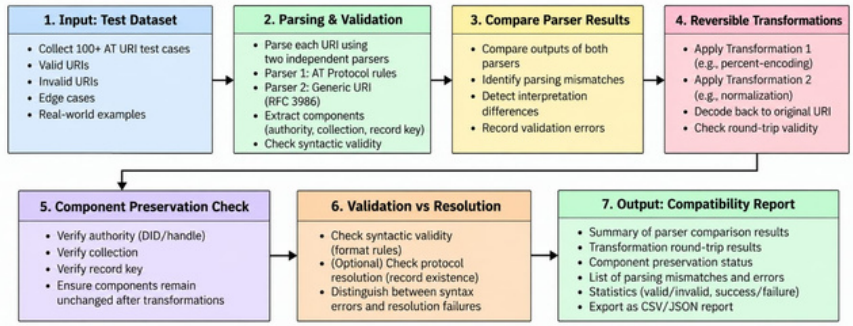



Figure: System Architecture & Workflow of AT URI Validator

# Data Flow Diagram

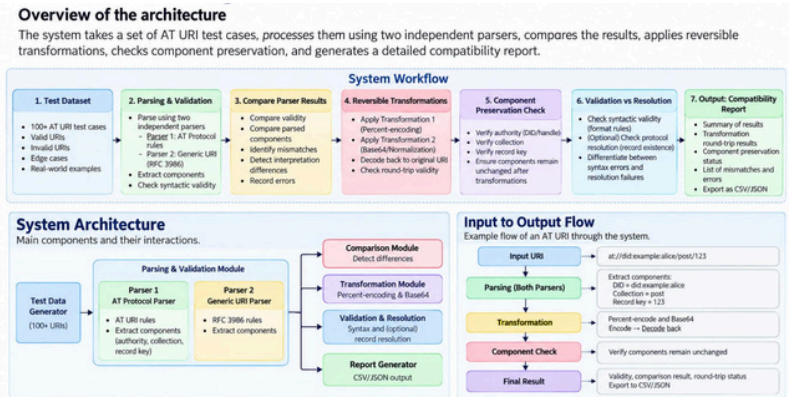



Figure: Data Flow Diagram (DFD)

# Deployment Timeline

**Phase 1: Project Setup**
- Set up Python and required libraries.
- Organize the project files.

**Phase 2: Dataset Preparation**
- Create 100+ valid, invalid, and edge-case AT URI test cases.

**Phase 3: Parser Development**
- Implement two parsers and compare their results against relevant URI rules.

**Phase 4: Transformation Testing**
- Test two reversible transformations.
- Verify preservation of identifier components.

**Phase 5: Testing and Reporting**
- Run automated tests and identify compatibility issues.
- Generate a CSV compatibility report.

**Phase 6: Final Demonstration**
- Upload the code and documentation to GitHub.
- Demonstrate the working prototype to the mentor.

# Bibliography I

[1] AT Protocol. “AT URI Scheme.”
https://atproto.com/specs/at-uri-scheme

[2] T. Berners-Lee, R. Fielding, and L. Masinter. “Uniform Resource Identifier (URI): Generic Syntax” RFC 3986, January 2005.
https://www.rfc-editor.org/rfc/rfc3986

Thank You for Your Attention!